AuxFlow: A Static Applicability Test for Fixpoint Convergence in Multi-Pass LaTeX Builds

Upayan Mazumder
School of Computer Science and Engineering (SCOPE)
Vellore Institute of Technology
Vellore, Tamil Nadu, India
upayan.mazumder2024@vitstudent.ac.in

Banupriya R
School of Computer Science and Engineering (SCOPE)
Vellore Institute of Technology
Vellore, Tamil Nadu, India
banupriya.r@vit.ac.in

Abstract*—Multi-pass document compilation in systems such as LaTeX resolves cross-references, generated tables of contents, and bibliographic citations through successive approximations across intermediate auxiliary files. Deployed build tools govern this iteration dynamically, evaluating heuristic termination criteria against hard-coded repetition caps without statically deriving pass requirements from document structure. Consequently, pathological documents exhibiting circular reference dependencies can induce non-termination or silently converge upon stale auxiliary state. This paper presents AuxFlow, a compiler front end and static analyzer for multi-pass LaTeX builds. AuxFlow formalizes auxiliary state dependencies over a declared language subset, lowering input sources into an instance-level Auxiliary Reference Graph (ARG) annotated with temporal pass lags. By propagating taint from resolved values to layout-governing control constructs, AuxFlow isolates content-amplifying cycles prior to execution, classifying documents into certified, benign-cyclic, amplifying, or unmodeled states and deriving provable pass bounds for certified inputs. Evaluating AuxFlow on an empirical corpus of 15 benchmark documents isolating nine published convergence mechanisms across 1050 containerized trials reveals that statically derived bounds strictly upper-bound empirical ground-truth requirements ($N \geq N^*$) across all 7 certified cases. AuxFlow identifies all 6 non-convergent documents before invoking the typesetting engine, operating with negligible analysis latency (1.3 ms median, 10.4 ms maximum). Finally, we characterize the structural boundaries where layout abstraction fails, examining the efficacy of post-hoc syntactic guards against variable-width oscillation.

***Index Terms*—LaTeX, build systems, static analysis, fixpoint iteration, attribute grammars**

I. Introduction

In document preparation systems such as LaTeX, typesetting documents with internal cross-references, tables of contents, or bibliographic databases is fundamentally an iterative fixpoint computation. During a primary compilation pass, the typesetting engine emits label definitions, section metadata, and citation keys to intermediate auxiliary files (such as `.aux` and `.toc`). Subsequent compilation passes re-read these records to reconcile forward references and settle page numbers. Because pagination shifts can alter referenced target values across successive runs, determining the required compilation depth is an intrinsic property of a document's cross-reference dependency structure.

TABLE I
Stopping tests and iteration ceilings in five deployed document compilation systems. Parameters derived from tool documentation and source code inspection (latexmk 4.88).

System	Stability test between passes	Limit	Failure diagnostic
latexmk	Timestamp, size, and MD5 file state	5	None
latexrun [5]	Input file digest invariance	None[a]	None
Tectonic	In-run read-then-written comparison	6	Modified filename
ConTeXt	MD5 digest of `.tuc` file	9	None
Typst	128-bit hash of layout iterations	5	Source spans and history

[a]No iteration limit is documented. Experimental runs aborted after 10 iterations (Section V-D).

In contemporary practice, document build drivers resolve this dependency structure dynamically rather than analytically. Build automation tools such as `latexmk` compare cryptographic checksums of generated files across consecutive iterations, capping execution at a default threshold of five runs [1]. Tectonic enforces a six-pass ceiling based on in-run file access tracking [2], ConTeXt iterates up to nine passes, and Typst caps internal layout resolution at five iterations [3] (Table I). Although dynamic checksumming functions reliably for typical documents, heuristic iteration caps exhibit sharp failure modes when processing non-convergent structures. Gaspar's canonical counterexample [4], for instance, constructs a loop bounded by a forward page reference that appends a page on every iteration, producing monotonic divergence. When dynamic tools exhaust their iteration caps on such inputs, three of the five widely deployed systems emit no diagnostic pinpointing the underlying causal dependencies.

Static pass scheduling itself rests on established theoretical foundations in compiler construction and circular attribute grammars. Seminal work by Bochmann [6] and Alblas [7] formalizes multi-pass attribute evaluation, Hecht and Ullman [8] bound iterative data-flow analysis rounds via flow-graph depth, and Jones [9] formulates evaluation over strongly connected components (SCCs) on condensed directed acyclic graphs. However, applying classical fixpoint theorems to document compilation exposes an essential mismatch: in programming language attribute grammars, the derivation tree remains invariant during attribute evaluation, whereas in document typesetting, auxiliary values directly alter page counts and text formatting, thereby mutating the underlying dependency graph across iterations.

AuxFlow investigates whether static influence analysis can determine when classical fixed-graph fixpoint theory safely applies to a document prior to compilation. Operating over a declared subset of LaTeX, AuxFlow establishes a conservative decision procedure for the finite-height criterion identified by Magnusson and Hedin [10], isolating dataflow paths that risk unbounded content generation while statically bounding the evaluation passes needed for convergent documents.

This paper makes the following contributions:

- **Formal Intermediate Representation and Influence Analysis:** We introduce the Auxiliary Reference Graph (ARG), an instance-level representation modeling auxiliary state interactions annotated with temporal pass lags. We develop a monotone taint propagation analysis that tracks when auxiliary values reach content-generating control constructs, distinguishing benign feedback from amplifying cycles prior to engine execution (Section III).
- **Comparative Empirical Evaluation:** We assemble an empirical corpus of 15 benchmark documents isolating nine published non-convergence mechanisms, systematically comparing five deployed convergence oracles against AuxFlow's static and verified execution models across 1050 containerized trials (Section V).
- **Characterization of Layout Feedback Limits:** We analyze the theoretical boundary where layout abstraction breaks down, evaluating how unmodeled extent feedback induces oscillation and assessing the efficacy of post-hoc syntactic guards on classification accuracy (Section VI).

II. Background and Related Work

A. The Auxiliary Round Trip

In standard LaTeX execution, a `\label` directive emits a `\newlabel` tuple containing a counter representation and page index to the primary auxiliary file (`.aux`). At `\begin {document}`, the kernel executes `\@input{\jobname .aux}` before opening the auxiliary stream for writing, ensuring that values written during run p become visible to reference commands (`\ref` , `\pageref`) only during run $p + 1$. We formalize this temporal delay as an invariant *pass lag* of exactly one. External utilities, such as BibTeX, process auxiliary citation entries between engine passes, introducing mediated dependency stages.

B. Multi-Pass Evaluation and Circular Attribute Grammars

Bochmann established conditions under which attribute grammars can be evaluated in alternating left-to-right passes [6]. Alblas subsequently distinguished *simple* multi-pass evaluation, where attribute types map to fixed passes, from *pure* multi-pass evaluation, where individual attribute instances may resolve in arbitrary passes [7]. While deciding pure multi-pass membership is inherently exponential and finding minimal pass assignments is NP-complete [11], instance-level tracking remains essential in document compilation because distinct labels within the same document exhibit divergent dependency depths. AuxFlow operates in Alblas's pure setting, deriving conservative upper bounds rather than asserting minimality.

Farrow formalized semantic evaluation over circular attribute grammars via successive approximations [12], while Jones optimized circular evaluation by partitioning dependency graphs into strongly connected components evaluated in topological order [9]. Magnusson and Hedin identified the necessary convergence conditions for circular reference attribute grammars: attributes on cyclic dependencies must form a complete partial order of finite height, and semantic transfer functions must remain monotonic [10]. While Magnusson and Hedin noted that their runtime evaluator left convergence unverified, AuxFlow provides a static decision procedure that checks the finite-height requirement within its modeled domain.

In higher-order attribute grammars, attribute evaluation can graft newly computed derivation trees into the syntax forest [13]. Krishnan and Van Wyk developed a static termination analysis for higher-order grammars using term-rewriting orderings over static production rules [14]. AuxFlow addresses a related self-generating phenomenon but operates on concrete document instances using reachability-based dataflow analysis over an empirical reference graph. Söderberg and Hedin [15] explored generalized evaluation algorithms for circular higher-order attributes, whereas AuxFlow focuses on pre-execution schedulability bounds.

C. Dataflow Bounds, Schedulability, and Build Systems

In classical data-flow analysis, Hecht and Ullman demonstrated that reverse post-order iteration over reducible flow graphs converges in $d + 2$ passes, where d represents loop connectedness [8], [16]. AuxFlow adopts an analogous decomposition: P passes propagate auxiliary values across lag boundaries, followed by a final pass to verify state invariance. In dataflow scheduling, Lee and Messerschmitt distinguished synchronous dataflow graphs with static

production rates from asynchronous systems with data-dependent production rates [17]. AuxFlow adapts this distinction to document processing: auxiliary values that alter only printed text represent fixed-rate computation, whereas values that govern loop bounds or conditional structures modulate content volume and threaten stability.

In build system theory, Mokhov et al. categorized build dependencies into static applicative tasks and dynamic monadic tasks [18]. Erdweg et al. implemented Pluto, an incremental build system featuring a fixpoint cycle handler for LaTeX and BibTeX, observing that sound static dependency detection in general TeX is undecidable due to dynamic macro expansion [19], [20]. AuxFlow addresses this challenge by restricting analysis to a declared subset, mapping unmodeled commands to opaque facts that safely force conservative execution. Mädje proved that document introspection stability is undecidable via Rice's theorem, prescribing constant iteration caps with localized error reporting [21]. AuxFlow complements this approach by providing a static decision procedure and schedule derivation for compliant documents.

D. Taxonomy of Non-Convergence Mechanisms

Published literature and community forensics document at least nine distinct non-convergence mechanisms in LaTeX (enumerated in Section V). Mechanisms M1 and M2 involve explicit programmatic control, where loops or conditionals branch on resolved page numbers. Mechanisms M3 through M7 involve implicit physical feedback, where the typeset width of a resolved numeral, float relocation heuristic, or marginal note shifts downstream paragraph line breaks and alters page boundaries. Mechanism M8 represents nondeterministic auxiliary state, while M9 represents spurious kernel warnings induced by active character expansions in section headings.

III. Approach

Figure 1 outlines the AuxFlow compilation pipeline. All stages from lexical analysis through build planning execute statically without invoking external TeX processes, ensuring that computational overhead remains bounded. Engine execution is confined strictly to the runtime verifier.

A. Declared Language Subset and Opaque Abstraction

AuxFlow's front end incorporates a category-code lexer, a recursive-descent parser, an inclusion forest representing multi-file projects, and symbol tables tracking labels, references, citations, counters, and loaded packages. Every AST node and symbol definition preserves source span coordinates (`file:line:col`) to support precise diagnostics.

The modeled subset encompasses structural sectioning (`\section` , `\chapter`), label declarations (`\label`), cross-reference lookups (`\ref` , `\pageref` , `\eqref`), generated lists (`\tableofcontents` , `\listoffigures`), bibliographic commands (`\cite` , `\bibliography`), counter arithmetic, and boolean conditionals (`\whiledo` , `\ifthenelse`). A curated registry of typesetting commands and styling packages (e.g., `\textbf` , `geometry`, `xcolor`) is treated as inert, as these constructs write no auxiliary state. Macro definitions are analyzed without body expansion: a user-defined macro is classified as inert only if every constituent token resolves to an inert primitive. Any unrecognized macro, dynamic inclusion, or unsupported package instantiates an OPAQUE fact node, triggering conservative fallback planning.

B. The Auxiliary Reference Graph (ARG)

The ARG represents document dependencies at the granularity of individual fact *instances*. Coarser abstractions that group dependencies by fact *kinds* collapse documents into trivial fixed-size lattices, eliminating the structural variation necessary for dataflow analysis.

As modeled in Fig. 2, each FACT records a unique identifier, an instance kind (e.g., LABEL_NUMBER, LABEL_PAGE, REF_USE, TOC_ENTRY, CONTENT_VOLUME), a symbolic key, an evaluation component (`exists` or `value`), and a mandatory source span. Directed DEP_EDGE records capture dependencies annotated with an integer `pass_lag`, a semantic classification (AUX_ROUNDTRIP, TOOL, LAYOUT, STRUCTURAL), and an explanatory diagnostic string.

C. Stable-Pass Numbering

For each fact f in an acyclic graph, its stable pass number $s(f)$ denotes the compilation pass after which its auxiliary state is guaranteed to remain invariant. Given predecessor dependencies $\mathrm{deps}(f)$ and edge lag function $\ell(g \to f)$, $s(f)$ is computed via the recurrence:

$$s(f) = \begin{cases} 1 & \mathrm{deps}(f) = \varnothing, \\ 1 + \max_{g \in \mathrm{deps}(f)} \big(s(g) + \ell(g \to f) - 1\big) & \text{otherwise.} \end{cases} \tag{1}$$

For standard auxiliary file interactions, $\ell = 1$, simplifying the recurrence to $1 + \max(s(g))$. Tool edges mediated by external utilities (such as BibTeX) are assigned an effective lag of zero within the engine recurrence, reflecting the fact that external tools execute synchronously between engine passes. Letting P denote the maximum stable pass across all auxiliary facts, AuxFlow derives a predicted engine pass bound of $N = P + 1$, where the additional pass guarantees verification against the converged state.

D. Content-Quantity Influence Propagation

In typical documents, generated list extents feed into a synthetic CONTENT_VOLUME fact representing global pagination. To prevent standard documents from collapsing into a single circular component, the ARG bifurcates label facts into `exists` and `value` components, deliberately decoupling reference values from downstream layout placement.

To detect true non-convergence, AuxFlow executes a forward taint analysis that identifies when reference values influence layout volume. The analysis seeds the worklist with facts read within loop bounds, conditional expressions,

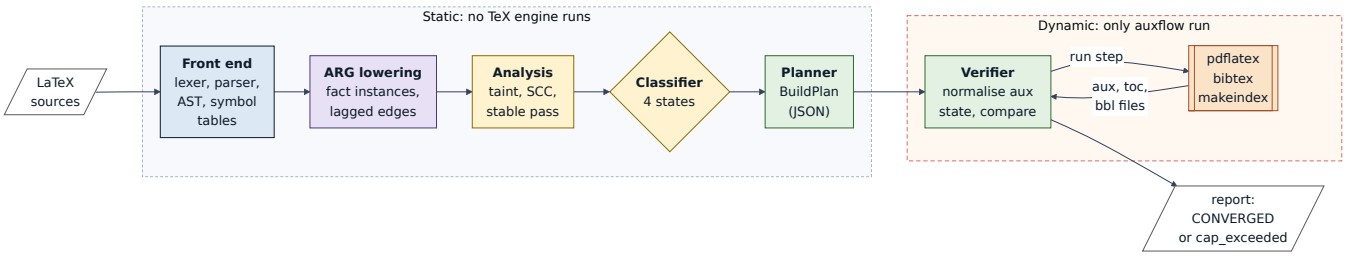


Fig. 1. The AuxFlow pipeline architecture. Static analysis executes entirely in-process without invoking external TEX binaries, producing an optimized build plan consumed by the runtime verification engine.

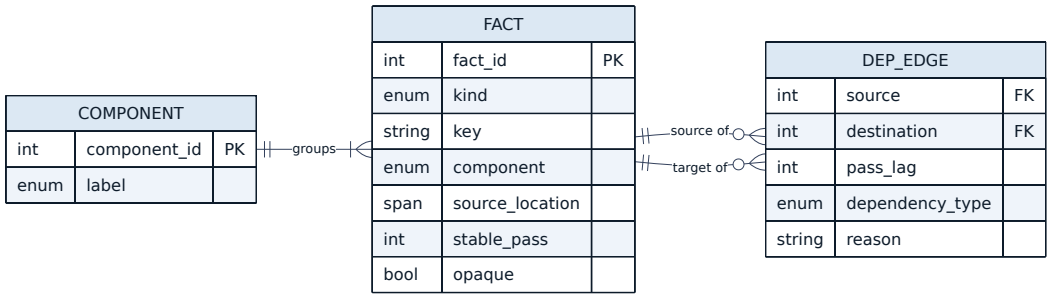


Fig. 2. Entity-relationship model of the Auxiliary Reference Graph. Input documents lower into concrete FACT instances and directed DEP_EDGE records. Strongly connected components derived via Tarjan's algorithm [22] govern document classification and subsequent build plan lowering.

or counter mutations governing control flow. Taint propagates monotonically along value-carrying dependency edges. For each tainted fact, AuxFlow injects a STRUCTURAL edge with lag one targeting CONTENT_VOLUME, recording the source span of the governing control structure.

Following taint injection, an iterative implementation of Tarjan's algorithm partitions the ARG into strongly connected components in $O(V + E)$ time. Components are categorized into three disjoint classes:

- AMPLIFYING: Components containing at least one internal STRUCTURAL edge, indicating that auxiliary values modulate their own generation rate.
- UNKNOWN_CYCLE: Components containing unmodeled or opaque facts.
- BENIGN: Circular components devoid of structural feedback, representing value-only recirculation that settles without altering document length.

This taint propagation over-approximates influence, treating any value dependency reaching a control construct as potentially non-convergent. This asymmetry ensures safety: while benign documents may occasionally receive an amplifying classification, no modeled amplifying cycle can escape detection.

E. Document Classification and Build Planning

Figure 3 details the document classification and lowering logic. The classifier evaluates components under strict precedence: AMPLIFYING takes precedence over UNKNOWN, followed by BENIGN_CYCLIC and CERTIFIED.

The build planner lowers classifications into a linear `BuildPlan`. To ensure safety, an invariant assertion guarantees that non-certified plans execute at least three engine runs. An AMPLIFYING document produces an empty plan that halts execution by default, requiring an explicit `--force` flag to execute under a defensive six-pass cap. For CERTIFIED documents, the planner optimizes execution by pruning unreachable files from auxiliary processing and scheduling BibTeX only when citations and bibliography commands co-exist.

F. Runtime Verification and State Normalization

Because LATEX kernel rerun warnings are fundamentally unsound on generated lists and vulnerable to spurious caption triggers, AuxFlow implements an independent runtime verification oracle.

Following each engine execution, the verifier extracts all `.aux`, `.toc`, `.lof`, `.lot`, and `.bbl` files, parsing their contents into canonical, order-independent record tuples (e.g., `("label", key, number, page)`). Formatting whitespace, comments, and boilerplate `\relax` tokens are discarded, while unrecognized records are preserved verbatim to ensure that unstable third-party data remains observable. The verifier evaluates set equality between the current state A_k and the preceding state A_{k-1} (Fig. 4). If a certified plan reaches its predicted bound without achieving state equality, the verifier dynamically extends execution up to a safety ceiling, logging a structured PREDICTION_MISS diagnostic.

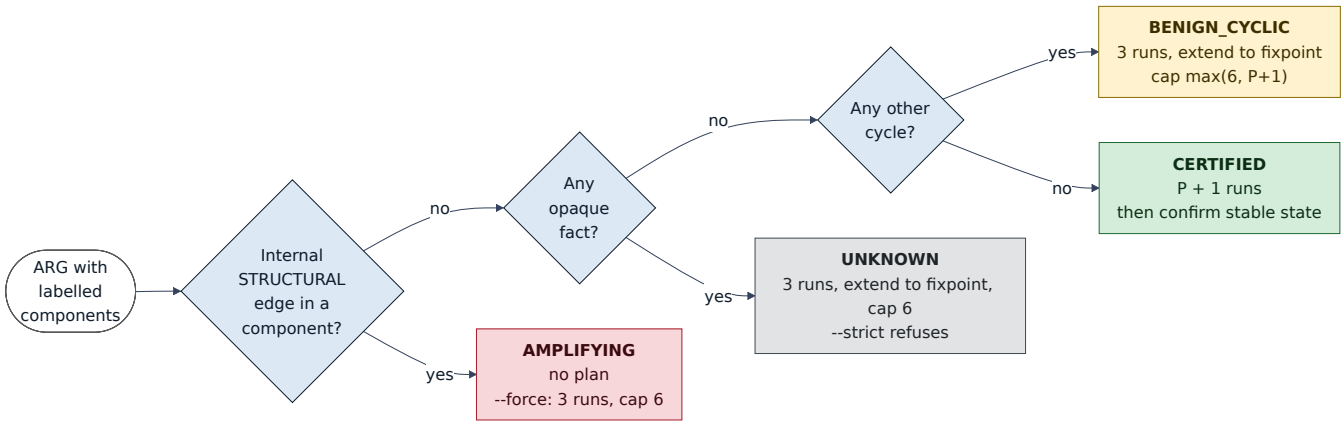


Fig. 3. Document classification hierarchy and plan generation policy. Certified documents execute a derived $P+1$ schedule, whereas uncertified inputs fall back to safe conservative floors with fixpoint verification.

Execution safeguards enforce a 600-second per-command timeout, disable shell escapes via `-no-shell-escape`, and reject auxiliary paths resolving outside the workspace root.

IV. Worked Example

To illustrate the mechanics of instance-level influence analysis, consider document `b08` from our benchmark suite, adapted from Gaspar's non-convergent loop construction [4]:

```
5 \whiledo{\value{generatedpage}
6           <\pageref{end:generated}}{%
7   \stepcounter{generatedpage}%
8   Record sheet occupies its own page.
9   \newpage }
10 End of the record.\label{end:generated}
```

In this structure, the `\whiledo` condition inspects the resolved page number of label `end:generated`. During pass k, the loop emits pages until the counter matches the prior pass value, placing the label on page $k+1$. The recorded page number increments by one on every run, inducing monotonic divergence.

AuxFlow lowers this source into the ARG depicted in Fig. 5. Because the `\whiledo` condition reads the page reference at line 5, the reference fact is identified as a taint seed. Taint propagation links this value to CONTENT_VOLUME via a STRUCTURAL edge. Together with the layout edge to LABEL_PAGE and the auxiliary round-trip edge back to the reference, these facts form a cyclic strongly connected component containing an internal structural dependency. AuxFlow classifies the document as AMPLIFYING prior to compilation, reporting the complete causal chain and source spans without running the engine. When executed under `--force`, AuxFlow monitors monotonic page expansion and halts at run six, reporting CAP_EXCEEDED (Fig. 6).

V. Evaluation

A. Experimental Design and Benchmark Corpus

Our empirical evaluation addresses four core research questions:

- **RQ1 (Classification Fidelity):** Does static cycle analysis accurately isolate modeled non-convergence mechanisms without flagging benign cyclic feedback?
- **RQ2 (Bound Safety):** Does the computed pass bound N strictly upper-bound empirical ground truth N^* across all certified documents?
- **RQ3 (Oracle Performance and Accuracy):** How do deployed convergence oracles compare in execution overhead, termination accuracy, and vulnerability to premature convergence?
- **RQ4 (Analysis Overhead):** What computational latency does static graph construction and taint propagation introduce relative to engine execution?

Figure 7 illustrates the evaluation framework. The benchmark suite comprises 15 documents designed to exercise specific dependency structures. To enforce methodological integrity, expected classifications and structural properties were pre-registered in version-controlled metadata files prior to analyzer implementation.

The benchmark compares seven execution strategies:

- **Baseline A (Fixed Schedule):** An unconditional three-pass schedule typical of conventional build scripts.
- **Baseline B (Engine Warning):** Dynamic re-execution governed by the LaTeX kernel's "Rerun to get cross-references right" warning.
- **Baseline C (Output Checksum):** Full-file auxiliary checksum comparison across consecutive runs (`latexmk` model).
- **Baseline C′ (Input Checksum):** Input file digest invariance detection (`latexrun` model).

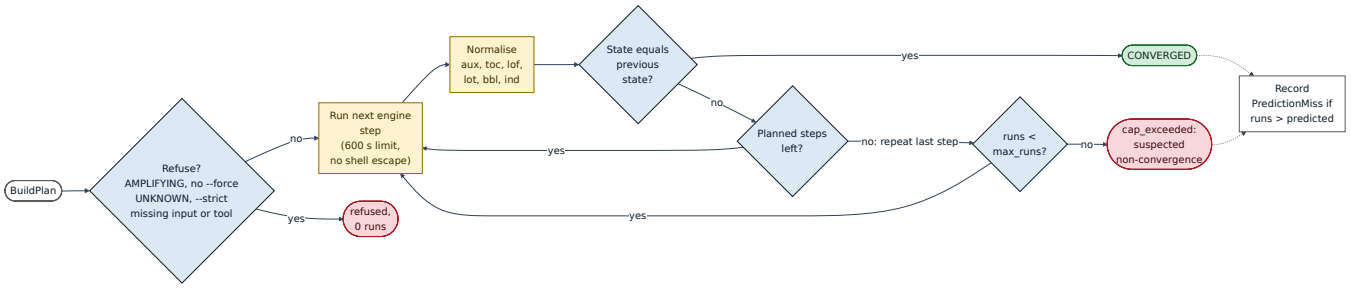
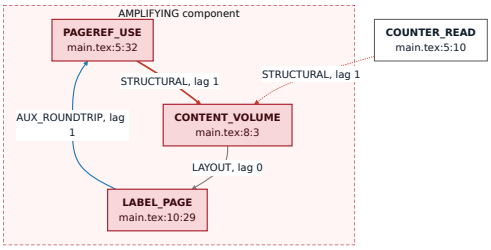


Fig. 4. The runtime verification workflow in `auxflow run`. Execution extends dynamically only if auxiliary state continues to mutate beyond planned pass bounds.




```
auxflow classify

$ auxflow classify main.tex
auxflow: AMPLIFYING CYCLE -- this document may not
  converge.

  A cross-reference value influences how much content the
    document generates,
  which in turn changes that value.

  cycle (3 facts):
    page reference to 'end:generated'  main.tex:5:32
      → amount of content the document generates
        main.tex:8:3  its value changes how much content
        is generated
      → page of label 'end:generated'  main.tex:10:29
        pagination sets page of label end:generated
      → page reference to 'end:generated'  main.tex:5:32

  likely cause:
    the page reference to 'end:generated' at
      main.tex:5:32: its value changes how much content
      is generated.

...
```

Fig. 5. Analysis of non-convergent document `b08`. Left: The lowered ARG, where taint analysis injects a STRUCTURAL edge from `PAGEREF_USE` to CONTENT_VOLUME. Right: Terminal output from `auxflow classify` detailing the causal cycle and exact source spans.

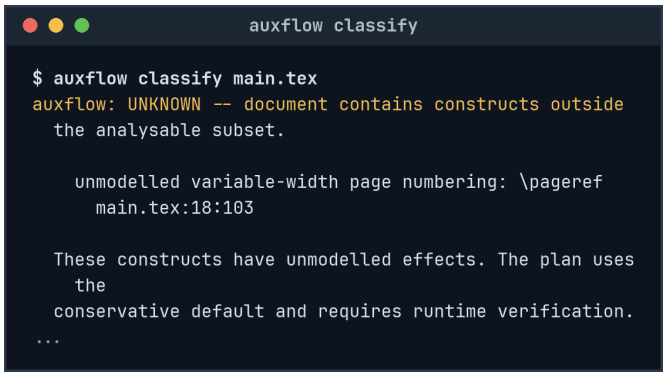
- **Baseline C″ (In-Run Comparison):** File-level read-then-written verification within a single pass (Tectonic model).
- **Baseline D (Static Analysis):** AuxFlow pre-compilation analysis operating without engine execution.
- **Baseline E (Verified Plan):** AuxFlow static build planning coupled with normalized runtime verification.

Ground Truth Derivation. Independent ground truth N^* is established by executing eight unconditional `pdflatex` passes without stopping rules. N^* represents the minimal run index $k < 8$ such that normalized auxiliary state and extracted text layers ($pdftotext - layout$) match identically across all subsequent passes up to run eight. Requiring full equality across all remaining runs is essential: alternating period-2 oscillations match coincidental state at even-numbered passes, producing spurious convergence if evaluated only against an isolated terminal pass. Documents failing this criterion are classified as divergent or oscillating.

Execution Environment. All dynamic trials executed inside isolated Docker containers built from `python:3.12-slim-bookworm` (digest prefix `93feb7fb6c24`), containing TeX Live 2022, `latexmk` 4.79, `latexrun`, Tectonic 0.15.0, and Typst 0.11.0. Containers were pinned to single CPU cores with networking disabled and fixed `SOURCE_DATE_EPOCH` timestamps. Each document-baseline pair was evaluated across ten independent repetitions (1050 total trials).

B. Classification Accuracy and Bound Safety (RQ1, RQ2)

Table II summarizes corpus classification results. AuxFlow successfully classifies `b08` as AMPLIFYING while



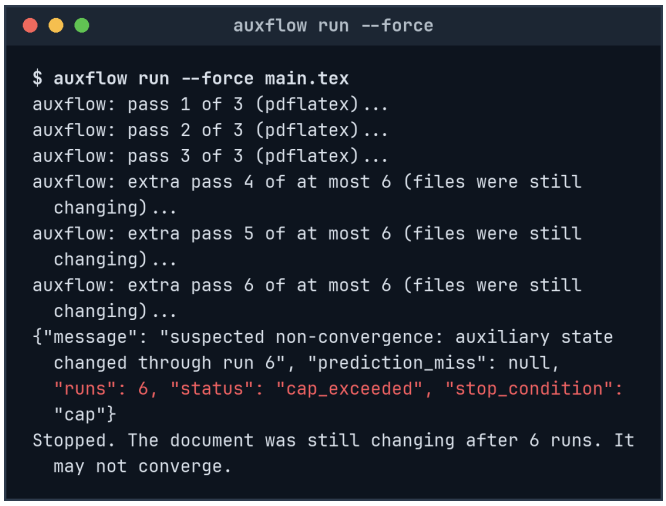


Fig. 6. CLI diagnostic captures. Left: `auxflow classify` flagging unmodeled variable-width numbering on `b11`. Right: `auxflow run --force` terminating at the six-pass ceiling on `b08`.

TABLE II
BENCHMARK CORPUS CHARACTERISTICS AND CLASSIFICATION RESULTS. N^* DENOTES EMPIRICAL GROUND TRUTH. "BEFORE" REFLECTS PRE-REGISTERED CLASSIFIER BEHAVIOR; "NOW" REFLECTS CURRENT ANALYZER BEHAVIOR INCORPORATING SYNTACTIC WIDTH GUARDS (BOLD INDICATES RECLASSIFICATION). N INDICATES PREDICTED BOUND.

ID	Target Mechanism	N^*	Before	Now	N
b01	no auxiliary facts	1	CERT	CERT	1
b02	backward `\ref`	2	CERT	CERT	2
b03	forward `\ref`	2	CERT	CERT	2
b04	TOC, then `\pageref`	2	CERT	CERT	3
b05	table of contents	3	CERT	CERT	3
b06	TOC, BibTeX, `\pageref`	3	CERT	CERT	4
b07	M7 footnote feedback	2	UNK	UNK	–
b08	M1 page-bounded loop	div.	AMPL	AMPL	–
b09	multi-chapter thesis	3	CERT	CERT	4
b10	M8 random auxiliary record	div.	UNK	UNK	–
b11	M3 numeral width	osc.	CERT	**UNK**	–
b12	M4 float, Roman numerals	osc.	CERT	**UNK**	–
b13	M6 `varioref` text	osc.	CERT	**UNK**	–
b14	M2 page-conditioned size	osc.	AMPL	AMPL	–
b15	M5 `ibid` citation style	3	UNK	UNK	–

TABLE III
ENGINE RUNS CONSUMED UNTIL TERMINATION (MEDIAN OF TEN TRIALS). SYMBOL † DENOTES UNSOUND TERMINATION (STALE STATE ACCEPTED OR NON-CONVERGENCE MISSED); `cap` DENOTES ITERATION CEILING REACHED; `ref.` INDICATES STATIC EXECUTION REFUSAL.

ID	N^*	A	B	C	C′	C″	E
b01	1	3	1	2	2	2	2
b02	2	3	2	2	2	2	2
b03	2	3	2	2	2	2	2
b04	2	3	2	2	2	2	2
b05	3	3	1†	3	3	3	3
b06	3	3	3	3	3	3	3
b07	2	3	2	2	2	2	ref.
b08	div.	3†	cap	cap	cap	cap	ref.
b09	3	3	3	3	3	3	3
b10	div.	3†	1†	cap	cap	cap	ref.
b11	osc.	3†	cap	cap	cap	cap	ref.
b12	osc.	3†	cap	cap	cap	cap	ref.
b13	osc.	3†	cap	cap	cap	cap	ref.
b14	osc.	3†	cap	cap	cap	cap	ref.
b15	3	3	1†	3	3	3	ref.

certifying the complex table-of-contents cascade in `b05`, demonstrating that the analysis discriminates content-modulating loops from benign structural recursion without false positives. Document `b14` is classified as AMPLIFYING due to conservative over-approximation: although the conditional selects between differently sized text without altering paragraph count, the taint framework safely flags the data-dependent branch.

Across all 7 certified documents, static pass bounds satisfy the safety invariant $N \geq N^*$. Bounds are tight ($N = N^*$) for 4 documents and overestimate by exactly one pass on 3 documents (`b04`, `b06`, `b09`), with zero instances of under-prediction (0 cases). Overestimation in `b06` and `b09` stems from conservative longest-path propagation across BibTeX tool boundaries. Furthermore, AuxFlow correctly isolates all 6 non-convergent documents prior to compilation, assigning two to AMPLIFYING and four to UNKNOWN.

C. Empirical Oracle Comparison (RQ3)

Table III and Fig. 8 document the engine executions consumed across baselines, while Table IV tallies overall oracle verdicts.

The empirical data highlights four key findings:

1. Systematic Blindness in Fixed Scheduling. Baseline A terminates after three passes across all documents, failing silently on all six non-convergent cases and providing no mechanism to detect instability.

2. Bidirectional Unsoundness in Engine Warnings. Baseline B demonstrates severe unreliability in both directions. On `b10`, where random tokens mutate auxiliary

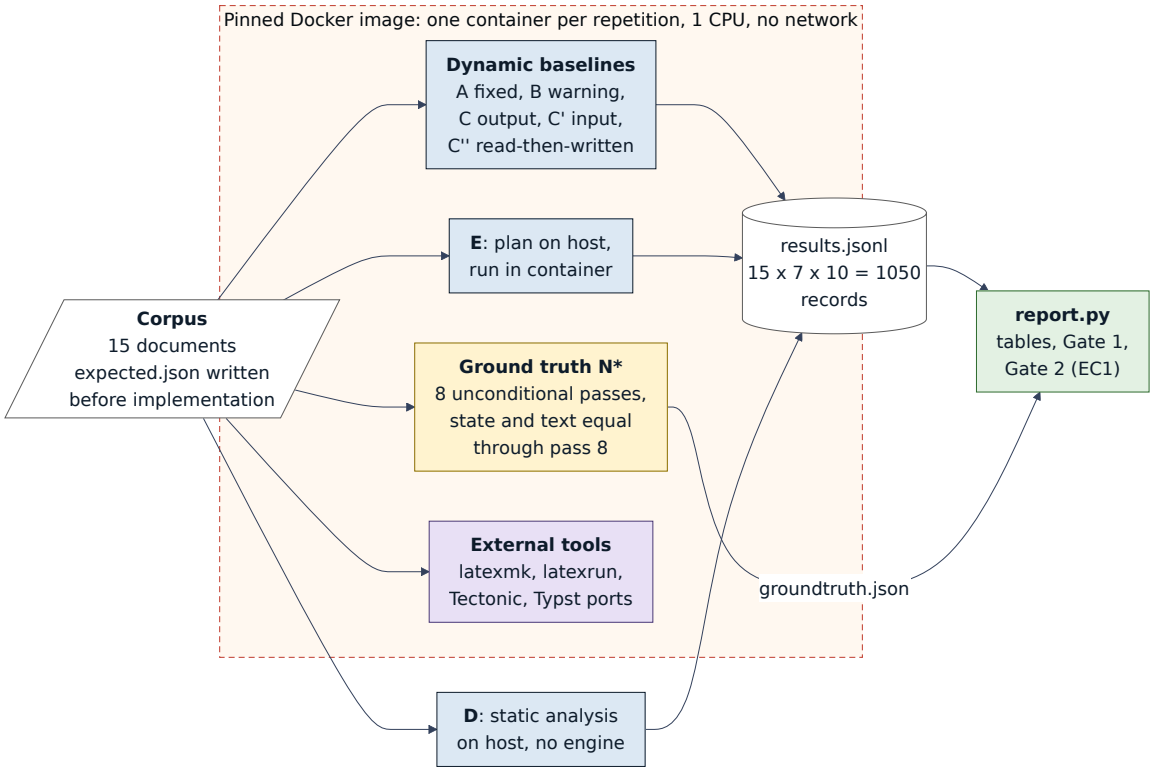


Fig. 7. Experimental test harness. Dynamic baselines A through E execute within isolated, resource-pinned Docker containers, benchmarked against independently derived ground-truth metrics N^*.

TABLE IV
SUMMARY OF TERMINATION VERDICTS ACROSS 15 CORPUS DOCUMENTS. DYNAMIC BASELINES A THROUGH E BENCHMARKED AGAINST STATIC ANALYSIS D.

Termination Outcome	A	B	C	C′	C″	E	D
Correct stop, of 9 convergent documents	9	7	9	9	9	7	7
Premature stop (stale output accepted)	0	2	0	0	0	0	0
Withheld by static refusal	0	0	0	0	0	2	2
Non-convergence flagged, of 6	0	5	6	6	6	6	6
Non-convergence accepted as converged	6	1	0	0	0	0	0
Engine runs spent on the 9 convergent documents	27	17	22	22	22	17	0

state, the engine emits no cross-reference warning, causing Baseline B to report convergence after a single run despite active divergence. Conversely, on `b05` and `b15`, Baseline B halts prematurely at run one ($N^* = 3$), accepting stale table-of-contents pagination because base LaTeX suppresses rerun warnings for structural list files [23].

3. Equivalence Among State Checksumming Oracles. Dynamic checksumming baselines C, C′, and C″ exhibit identical termination behavior across the corpus, spending 17 runs across the seven certified documents. Although literature suggests that input-comparison (C′) and in-run access tracking (C″) can eliminate the redundant confirmation pass required by output comparison (C), this theoretical advantage did not manifest on our benchmark documents.

4. Efficiency and Safety in Verified Static Planning. Baseline E matches the efficiency of the best dynamic detectors on all certified documents (17 runs total vs. 21 for Baseline A). On uncertified documents, Baseline E refuses execution upfront for eight cases (the six non-convergent documents plus opaque cases `b07` and `b15`), avoiding futile engine iteration. On trivial document `b01` ($N^* = 1$), Baseline E requires two runs because the LaTeX kernel unconditionally emits an internal `\@abspage@last` token, preventing the auxiliary state from evaluating to empty on pass one. This structural property triggers a recorded prediction miss, though the underlying static prediction remains correct.

D. Cross-Validation Against External Systems (RQ3)

To corroborate our non-convergence findings, we evaluated deployed versions of `latexmk`, `latexrun`, and Tectonic on the same corpus (Table V). Both `latexmk` and `latexrun` fail to converge on all six non-convergent documents, validating that non-convergence is an intrinsic property of these document structures rather than an artifact of AuxFlow's harness. Tectonic compiles only `b01` and `b08` in offline mode due to sandboxed package retrieval constraints;

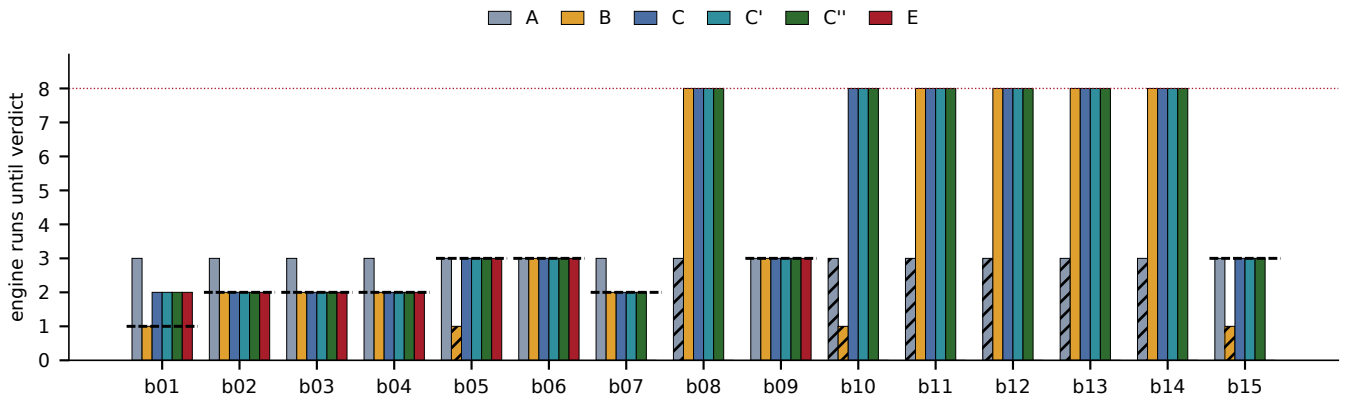


Fig. 8. Compilation passes across benchmark documents and baselines. Dashed horizontal markers denote ground truth N^*. Hatched bars highlight unsound convergence; missing bars in Baseline E reflect upfront static refusal; the dotted line indicates the 8-pass ceiling.

TABLE V
EXTERNAL TOOL CONVERGENCE ON CHALLENGING CORPUS DOCUMENTS. ALL UNLISTED CORPUS DOCUMENTS CONVERGE SUCCESSFULLY ACROSS ALL TOOLS. METRIC `n/a` DENOTES OFFLINE COMPILATION LIMITATIONS IN TECTONIC.

ID	latexmk	latexrun	Tectonic
b08	no	no	cap
b10	no	no	n/a
b11	no	no	n/a
b12	no	no	n/a
b13	no	no	n/a
b14	no	no	n/a
b15	yes	no	n/a

on `b08`, it flags that a rerun is needed before aborting at its six-pass ceiling. Furthermore, our hand-ported Typst implementation of `b08` halts with an explicit compiler error (`loop seems to be infinite`), providing independent compiler-level confirmation of amplifying divergence.

E. Computational Overhead (RQ4)

Static analysis executes with high efficiency, requiring a median of 1.3 ms and a maximum of 10.4 ms across all documents. Even on the realistic thesis document (`b09`), which lowers to 91 facts and 73 edges, analysis completes in 7.5 ms. Because full typesetting runs require multiple seconds per document, the static planning overhead remains negligible.

VI. EVOLUTION OF LAYOUT FEEDBACK GUARDS

During initial benchmark evaluations, documents `b11`, `b12`, and `b13` were certified by the analyzer. This constituted a planned false negative: because their cyclic instability originates from the typeset physical width of resolved page numerals shifting paragraph line breaks rather than explicit control-flow loops, taint analysis seeded no structural edges.

To mitigate this vulnerability, we incorporated three post-hoc syntactic guards into the IR lowering stage:

1) Reference lookups operating under variable-width page numbering (`\pagenumbering {roman}`, `Roman`, `alph`, `Alph`) instantiate an OPAQUE fact, as numeral width fluctuations can perturb line breaking.
2) Dynamic reference directives provided by packages such as `varioref` (`\vref` , `\vpageref`) unconditionally generate OPAQUE facts.
3) Standard front-matter commands and formatting packages are explicitly whitelisted as inert, preventing false UNKNOWN classifications on common styling constructs.

These guards reclassify documents `b11` through `b13` from CERTIFIED to UNKNOWN (Table II). While this eliminates empirical bound violations, methodological transparency requires acknowledging three limitations:

- The reclassification causes Gate 1 verification to report failure against pre-registered baseline expectations.
- The intervention is post-hoc, meaning the corpus no longer provides an independent evaluation of the guard.
- The heuristic pattern-matches syntax rather than modeling font metrics. Layout oscillation occurring under Arabic numbering or custom macros remains unmodeled and would escape detection.

VII. THREATS TO VALIDITY AND LIMITATIONS

Corpus Representativeness. The benchmark suite contains fifteen documents, most of which are synthetic micro-benchmarks designed to isolate specific compilation mechanisms. While `b09` mirrors a production academic thesis, broader validation on large-scale open-source document corpora is necessary to assess macro-level package compatibility.

Methodological Author Bias. To mitigate potential bias inherent in authoring both the static analyzer and the benchmark corpus, structural properties and expected classifications were codified in version control prior to parser implementation.

The Layout Invariance Abstraction (Assumption A2). AuxFlow adopts the pragmatic modeling assumption that resolving reference values does not perturb global pagination. In physical typesetting, replacing placeholder glyphs (`??`) with multi-digit numerals can alter line breaks. Modeling font metrics and line-breaking algorithms would cause the reference graph to degenerate into a monolithic circular component; AuxFlow accepts this abstraction barrier, measuring the resulting unsoundness openly.

Conflation of Divergence and Oscillation. The AMPLIFYING classification combines monotonic divergence (violating lattice finite-height) and periodic oscillation (violating monotonicity). While separating these modes is theoretically appealing, detecting oscillation without evaluating geometric font metrics remains intractable.

Toolchain and Engine Drift. Containerized evaluations executed under TeX Live 2022, whereas host environments utilized TeX Live 2026. During harness development, residual auxiliary files in a corpus directory caused premature termination in baseline B; clearing these artifacts restored deterministic evaluation.

VIII. Conclusion

Multi-pass document compilation represents a dynamic fixpoint computation where auxiliary data can mutate the underlying dependency graph. AuxFlow demonstrates that static influence analysis over a declared LaTeX subset can reliably verify the preconditions of classical fixpoint convergence. Across our empirical corpus, AuxFlow correctly bounded all certified documents, isolated all non-convergent inputs prior to execution, and imposed negligible computational latency.

Future work will explore integrating bounding box approximations to detect font-metric feedback without full rendering, expanding the modeled package surface, and formally bisecting amplifying cycles into divergent and oscillating classifications.

Availability

AuxFlow is distributed as open-source software under the MIT license, available on GitHub [24] and PyPI (`auxflow`). The complete benchmark suite, container configurations, and execution harness are archived in the repository; running `python paper/make_results.py` deterministically regenerates all empirical tables and figures presented in this paper.

References

[1] J. Collins, "latexmk: automatic LaTeX document generation," CTAN, https://ctan.org/pkg/latexmk, version 4.88 manual and source.

[2] Tectonic Typesetting Project, "Tectonic: a modernized, complete, self-contained TeX/LaTeX engine," https://github.com/tectonic-typesetting/tectonic, version 0.15.0.

[3] Typst contributors, "Typst: a markup-based typesetting system," https://github.com/typst/typst, version 0.11.0.

[4] Gaspar, "Does not suffice to run LaTeX a finite number of times to get cross-references right," *TUGboat*, vol. 35, no. 1, p. 16, 2014.

[5] A. Clements, "latexrun: a LaTeX wrapper that fixes the bad parts of LaTeX," https://github.com/aclements/latexrun.

[6] G. V. Bochmann, "Semantic evaluation from left to right," *Communications of the ACM*, vol. 19, no. 2, pp. 55–62, 1976.

[7] H. Alblas, "A characterization of attribute evaluation in passes," *Acta Informatica*, vol. 16, no. 4, pp. 427–464, 1981.

[8] M. S. Hecht and J. D. Ullman, "A simple algorithm for global data flow analysis problems," *SIAM Journal on Computing*, vol. 4, no. 4, pp. 519–532, 1975.

[9] L. G. Jones, "Efficient evaluation of circular attribute grammars," *ACM Transactions on Programming Languages and Systems*, vol. 12, no. 3, pp. 429–462, 1990.

[10] E. Magnusson and G. Hedin, "Circular reference attributed grammars: their evaluation and applications," *Science of Computer Programming*, vol. 68, no. 1, pp. 21–37, 2007.

[11] K.-J. Räihä and E. Ukkonen, "Minimizing the number of evaluation passes for attribute grammars," *SIAM Journal on Computing*, vol. 10, no. 4, pp. 772–786, 1981.

[12] R. Farrow, "Automatic generation of fixed-point-finding evaluators for circular, but well-defined, attribute grammars," *SIGPLAN Notices*, vol. 21, no. 7, pp. 85–98, 1986.

[13] H. H. Vogt, S. D. Swierstra, and M. F. Kuiper, "Higher order attribute grammars," in *Proc. ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*, 1989.

[14] L. Krishnan and E. V. Wyk, "Termination analysis for higher-order attribute grammars," in *Software Language Engineering (SLE 2012)*, ser. LNCS, vol. 7745, 2012, pp. 44–63.

[15] E. Söderberg and G. Hedin, "Circular higher-order reference attribute grammars," in *Software Language Engineering (SLE 2013)*, ser. LNCS, vol. 8225, 2013, pp. 302–321.

[16] J. B. Kam and J. D. Ullman, "Global data flow analysis and iterative algorithms," *Journal of the ACM*, vol. 23, no. 1, pp. 158–171, 1976.

[17] E. A. Lee and D. G. Messerschmitt, "Static scheduling of synchronous data flow programs for digital signal processing," *IEEE Transactions on Computers*, vol. C-36, no. 1, pp. 24–35, 1987.

[18] A. Mokhov, N. Mitchell, and S. Peyton Jones, "Build systems à la carte," *Proceedings of the ACM on Programming Languages*, vol. 2, no. ICFP, 2018.

[19] S. Erdweg, M. Lichter, and M. Weiel, "A sound and optimal incremental build system with dynamic dependencies," in *Proc. ACM International Conference on Object-Oriented Programming, Systems, Languages, and Applications (OOPSLA)*, 2015.

[20] S. Erdweg and K. Ostermann, "Featherweight TeX and parser correctness," in *Software Language Engineering (SLE 2010)*, 2010.

[21] L. Mädje, "A programmable markup language for typesetting," Master's thesis, Technische Universität Berlin, Sep. 2022.

[22] R. Tarjan, "Depth-first search and linear graph algorithms," *SIAM Journal on Computing*, vol. 1, no. 2, pp. 146–160, 1972.

[23] H. Oberdiek, "The rerunfilecheck package: checksum based rerun checks on auxiliary files," CTAN, https://ctan.org/pkg/rerunfilecheck.

[24] U. Mazumder, "AuxFlow: static analysis for multi-pass document compilation," https://github.com/upayanmazumder/auxflow, release 0.3.0; PyPI package `auxflow`.